

Chapitre : Arbre de Décisions

Data Mining

DJIRAIKINAN TITEMBAYE

2026

Sommaire

- Généralités
- Architecture
- Exemples
- Algorithmes et cas concrets
- Sur-apprentissage et élagage

Problème

Une banque souhaite décider si un client obtient un crédit.

- Question 1 : revenu suffisant ?
- Question 2 : historique de paiement sain ?
- Question 3 : ancienneté du compte.

L'arbre transforme ces questions en une suite de tests automatiques.

Principes fondamentaux

- **Intuition** : Une technique populaire et facile à interpréter.
- **Type de tâche** : Apprentissage supervisé pour la **classification** (variable cible catégorielle) et la **Régression** (variable cible continue).
- **Positionnement** : À la frontière entre le **descriptif** et le **prédictif** (classification hiérarchique supervisée).
- **Concurrence** : Souvent comparée à la *régression logistique*

Avantages méthodologiques

- **Interprétabilité** : Une lisibilité immédiate pour les décideurs (modèle *boîte blanche*).
- **Flexibilité** : Adaptation naturelle aux types de données mixtes (numériques et catégorielles).
- **Exploration** : Très efficace pour identifier les variables les plus discriminantes en début d'étude.
- **Modularité** : Base aux algorithmes d'ensemble (*Random Forest, Gradient Boosting*).

Avantages méthodologiques

- **Interprétabilité** : Une lisibilité immédiate pour les décideurs (modèle *boîte blanche*).
- **Flexibilité** : Adaptation naturelle aux types de données mixtes (numériques et catégorielles).
- **Exploration** : Très efficace pour identifier les variables les plus discriminantes en début d'étude.
- **Modularité** : Base aux algorithmes d'ensemble (*Random Forest, Gradient Boosting*).

Note

Visualiser un arbre permet d'expliquer la logique de décision sans recourir à un formalisme mathématique complexe.

Structure et Connectivité

- **Directionnel** : Les branches sont orientées du *nœud parent* vers le *nœud enfant*.
- **Unicité** : Chaque nœud possède un **unique parent** (sauf la racine).
- **Arborescence** : Un nœud peut avoir 0 à n enfants.
- **nœuds de décision** : divise les données en fonctions d'un test

Structure et Connectivité

- **Directionnel** : Les branches sont orientées du *nœud parent* vers le *nœud enfant*.
- **Unicité** : Chaque nœud possède un **unique parent** (sauf la racine).
- **Arborescence** : Un nœud peut avoir 0 à n enfants.
- **nœuds de décision** : divise les données en fonctions d'un test

Le Nœud Racine

Point d'entrée de l'arbre, c'est le seul nœud qui n'a **aucun parent**.

Les Nœuds Feuilles

Points terminaux de l'arbre, ce sont les nœuds qui n'ont **aucun enfant**.

Structure et Connectivité

- **Directionnel** : Les branches sont orientées du *nœud parent* vers le *nœud enfant*.
- **Unicité** : Chaque nœud possède un **unique parent** (sauf la racine).
- **Arborescence** : Un nœud peut avoir 0 à n enfants.
- **nœuds de décision** : divise les données en fonctions d'un test

Le Nœud Racine

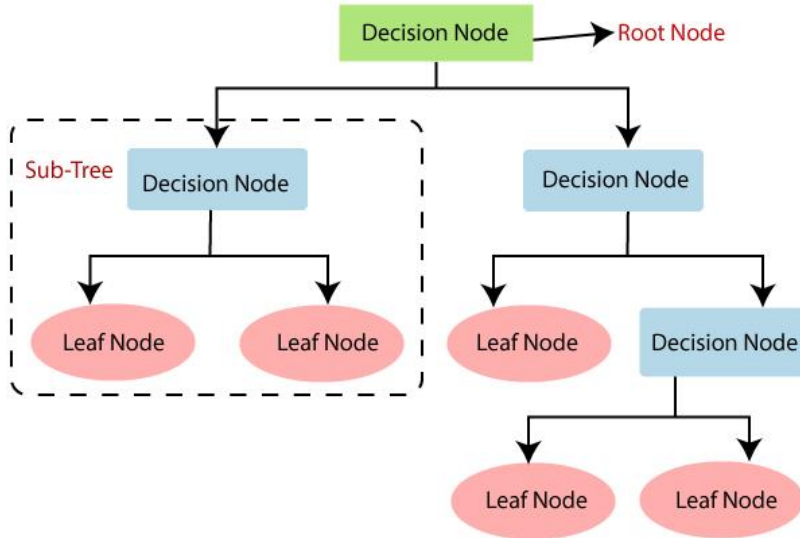
Point d'entrée de l'arbre, c'est le seul nœud qui n'a **aucun parent**.

Les Nœuds Feuilles

Points terminaux de l'arbre, ce sont les nœuds qui n'ont **aucun enfant**.

Note : Contrairement à un réseau/graphe général, l'arbre ne contient pas de cycles.

Architecture



Exemple 1 :classification

Objectif

Déterminer si un prêt peut être accordé à un client selon son profil.

Revenu	Historique	Ancienneté	Décision
25k	mauvais	1 an	non
30k	bon	2 ans	oui
40k	bon	3 ans	oui
50k	mauvais	1 an	non
20k	mauvais	1 an	non
45k	bon	4 ans	oui
35k	bon	2 ans	oui
28k	mauvais	1 an	non
55k	bon	5 ans	oui
22k	bon	3 ans	oui

Exemple 1 : Classification

- **Test à la racine** : Revenu $\geq$ 35k ?

Gauche : Revenu $<$ 35k (5 clients)

Revenu	Historique	Ancienneté	Décision
25k	mauvais	1 an	non
30k	bon	2 ans	oui
20k	mauvais	1 an	non
28k	mauvais	1 an	non
22k	bon	3 ans	oui

Droite : Revenu $\geq$ 35k (5 clients)

Revenu	Historique	Ancienneté	Décision
40k	bon	3 ans	oui
50k	mauvais	1 an	non
45k	bon	4 ans	oui
35k	bon	2 ans	oui
55k	bon	5 ans	oui

Exemple 1 : Classification

- Test Sous arbre A
- Pour revenu $< 35k$: Test sur Historique de paiement

Mauvaise Historique

Revenu	Historique	Ancienneté	Décision
25k	mauvais	1 an	non
20k	mauvais	1 an	non
28k	mauvais	1 an	non

Bonne historique Historique

Revenu	Historique	Ancienneté	Décision
30k	bon	2 ans	oui
22k	bon	3 ans	oui

Exemple 1 : Classification

- Test sous-arbre B
- Pour revenu $\geq 35k$: Test sur Ancienneté ≥ 2 ans

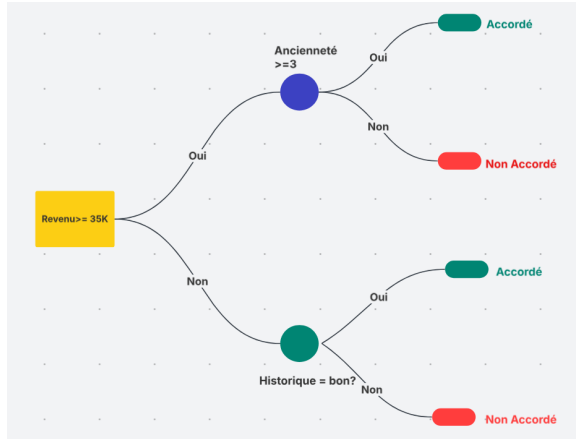
Mauvaise Historique

Revenu	Historique	Ancienneté	Décision
50k	mauvais	1 an	non

Bonne historique Historique

Revenu	Historique	Ancienneté	Décision
40k	bon	3 ans	oui
45k	bon	4 ans	oui
35k	bon	2 ans	oui
55k	bon	5 ans	oui

Exemple 1 : Classification



Exemple 2 : Régression

Objectif

Estimer le prix d'une maison selon ses caractéristiques.

- Variables prédictives : superficie, nombre de chambres, quartier
- Variable cible : prix (continue en milliers d'euros)

Superficie (m ²)	Chambres	Quartier	Prix (k€)
80	2	Centre	250
120	3	Périphérie	180
100	3	Centre	300
90	2	Périphérie	160
150	4	Centre	400
110	3	Périphérie	200
85	2	Centre	270
130	4	Périphérie	220

Exemple 2 : Régression

- **Test à la racine** : Superficie $\geq 100 \text{ m}^2$?

Gauche : Superficie $< 100 \text{ m}^2$

Superficie	Chambres	Quartier	Prix
80	2	Centre	250
90	2	Périphérie	160
85	2	Centre	270
Moyenne :			227 k€

Droite : Superficie $\geq 100 \text{ m}^2$

Superficie	Chambres	Quartier	Prix
120	3	Périphérie	180
100	3	Centre	300
150	4	Centre	400
110	3	Périphérie	200
130	4	Périphérie	220
Moyenne :			260 k€

Exemple 2 : Régression

- Test sous-arbre gauche : Quartier = Centre ?

Quartier Centre ($< 100 \text{ m}^2$)

Superficie	Chambres	Quartier	Prix
80	2	Centre	250
85	2	Centre	270
Moyenne :			260 k€

Quartier Périphérie ($< 100 \text{ m}^2$)

Superficie	Chambres	Quartier	Prix
90	2	Périphérie	160
Moyenne :			160 k€

Exemple 2 : Régression

- Test sous-arbre droite : Chambres ≥ 4 ?

Chambres < 4 ($\geq 100 \text{ m}^2$)

Superficie	Chambres	Quartier	Prix
120	3	Périphérie	180
100	3	Centre	300
110	3	Périphérie	200
Moyenne :			227 k€

Chambres ≥ 4 ($\geq 100 \text{ m}^2$)

Superficie	Chambres	Quartier	Prix
150	4	Centre	400
130	4	Périphérie	220
Moyenne :			310 k€

Exemple 2 : Régression - Arbre final

- Chaque feuille prédit la moyenne des prix dans ce groupe
- Pour une nouvelle maison :
 - Si superficie $< 100 \text{ m}^2$ et quartier Centre : prédiction = 260 k€
 - Si superficie $< 100 \text{ m}^2$ et quartier Périphérie : prédiction = 160 k€
 - Si superficie $\geq 100 \text{ m}^2$ et chambres < 4 : prédiction = 227 k€
 - Si superficie $\geq 100 \text{ m}^2$ et chambres ≥ 4 : prédiction = 310 k€

- Trois familles d'arbres dominant : ID3, C4.5 et CART.
- Elles diffèrent par le critère de division et le traitement des données.
- Chaque algorithme cherche à choisir le test le plus pertinent au niveau du nœud.
- C'est cette sélection qui conditionne la qualité finale de l'arbre.

Iterative Dichotomiser 3

ID3 utilise le gain d'information pour sélectionner l'attribut à chaque nœud.

- Fonctions uniquement sur des attributs discrets.
- Calcule l'entropie du nœud avant et après chaque division.
- La meilleure division maximise la réduction d'entropie.
- Arrêt quand un nœud est pur ou qu'aucun gain n'est disponible.

C4.5 : l'évolution d'ID3

- C4.5 gère les attributs numériques par test de seuils.
- Il traite les valeurs manquantes par répartition probabiliste.
- Il utilise le gain ratio pour corriger le biais des attributs à nombreuses modalités.
- Il produit des arbres plus robustes en conditions réelles.

CART : arbres binaires et régression

- Produit uniquement des splits binaires.
- Utilise l'indice de Gini pour la classification.
- Utilise l'erreur quadratique moyenne pour la régression.
- Base de modèles d'ensemble comme les forêts aléatoires.

Caractéristique	ID3	C4.5	CART
Type d'arbre	Multi-branches	Multi-branches	Binaire
Critère	Gain d'info	Ratio de gain	Gini / MSE
Attributs	Catégoriels	Tous	Tous
Valeurs manquantes	Non gérées	Gérées	Gérées
Tâche	Classification	Classification	Class. + Régression

Entropie : mesure de l'incertitude

$$H(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

- S : ensemble de données considéré
 - c : nombre de classes dans S
 - p_i : proportion d'exemples de la classe i dans S ($p_i = \frac{|S_i|}{|S|}$)
-
- L'entropie quantifie l'impureté d'un ensemble S : elle est maximale (1 pour 2 classes) lorsque les classes sont équilibrées (ex. : 50% / 50%).
 - Elle atteint zéro lorsque l'ensemble est pur (une seule classe représentée).
 - Constitue la base du gain d'information dans les arbres de décision.

Gain d'information

$$\text{Gain}(S, A) = H(S) - \sum_{v \in \text{Valeurs}(A)} \frac{|S_v|}{|S|} H(S_v)$$

- S : ensemble de données parent
 - A : attribut candidat pour le split
 - v : valeur possible de l'attribut A
 - $|S_v|$: nombre d'exemples dans le sous-ensemble où $A = v$
 - $|S|$: taille totale de S
 - $H(S_v)$: entropie du sous-ensemble S_v
-
- Mesure la réduction d'entropie apportée par le split sur l'attribut A .
 - Un gain élevé indique un attribut discriminant : il maximise la séparation des classes.
 - Sélection récursive de l'attribut avec le gain maximal pour construire l'arbre.
 - Favorise les attributs avec de nombreuses valeurs (risque de sur-apprentissage).

Exercice

Soit le dataset suivant représentant 6 clients d'une banque. L'objectif est de prédire la cible **Approuvé**. Construire l'arbre de décision en se basant sur ID3.

ID	Revenu	Historique	Stable	Approuvé
1	Élevé	Bon	Oui	Oui
2	Faible	Mauvais	Non	Non
3	Élevé	Mauvais	Oui	Oui
4	Moyen	Bon	Non	Oui
5	Faible	Bon	Non	Non
6	Moyen	Mauvais	Oui	Non

Indice de Gini

$$G(S) = 1 - \sum_{i=1}^c p_i^2$$

- S : ensemble de données considéré
 - c : nombre de classes dans S
 - p_i : proportion d'exemples de la classe i dans S ($p_i = \frac{|S_i|}{|S|}$)
-
- Utilisé par l'algorithme CART pour la classification binaire et multiclasse.
 - Gini = 0 pour un ensemble pur (une seule classe), Gini = 0.5 pour un équilibre parfait (2 classes à 50%).
 - Optimise l'homogénéité des sous-ensembles.

Processus

On construit l'arbre récursivement : on divise, puis on divise encore, jusqu'à ce que la structure soit satisfaisante.

- Partir de la racine avec toutes les observations.
- Tester les attributs disponibles et choisir la meilleure séparation.
- Appliquer la même logique aux sous-ensembles.
- S'arrêter quand la qualité ne s'améliore plus.

Un arbre peut parfaitement classer les exemples d'entraînement, mais rater les nouveaux clients.

- Trop de branches = trop de détails.
 - Certaines divisions capturent le bruit plutôt que le signal.
 - Résultat : moins de robustesse en production.
-
- Limiter la profondeur maximale.
 - Exiger un nombre minimum d'observations par feuille.
 - Préférer un arbre simple si l'écart de performance est faible.

Élagage préventif

- Arrêter la croissance avant qu'elle ne devienne trop importante.
- Contrôler la profondeur maximale.
- Fixer un nombre minimum d'exemples par feuille.
- Éviter la création de branches peu fiables.

Élagage postérieur

- Construire d'abord l'arbre complet.
- Puis supprimer les branches qui n'améliorent pas la validation.
- On peut utiliser une validation croisée pour décider.

Questions ?